



Certificates and Cryptographic Methods

CEN429 Secure Programming — Week 10

Asst. Prof. Dr. Uğur CORUH · 20.11.2026

Today's Plan (3 Hours)

Hour	Section	Topic
1	1–3	Algorithm/key selection · block cipher modes/padding · MAC/HMAC
2	4–6	RSA/ECC (OAEP/PSS) · digital signature · Diffie–Hellman
3	7–13	PKI · X.509 · building a chain with OpenSSL · CRL/OCSP · HSM/PKCS#11 · post-quantum · project

Learning outcomes (LO.2 / LO.4): choose the right algorithm, mode, padding, and key length · spot the pitfalls of signatures and key exchange · correctly validate a certificate chain

In cryptography, mistakes are usually not in the algorithm but in the **usage**: the wrong mode, the wrong padding, an unverified signature, an unchecked chain.

- **Symmetric and asymmetric encryption** — symmetric encryption runs fast with a single shared key; asymmetric encryption uses a public/private key pair, making key distribution easy but slow (**Week 3**)
- **AEAD** — authenticated encryption gives confidentiality and integrity together, in a single call (**Week 3**)
- **Digest, MAC, and digital signature** — a digest is a one-way fingerprint; a MAC proves a message hasn't changed with a shared key, a signature does the same with a public/private key pair (**Week 3**)
- **Forward secrecy** — deleting a session's derived key after use, so that even if today's key leaks, past sessions cannot be decrypted (**Week 3**)
- **TLS certificate validation** — the client checking a server certificate's chain, validity, intended usage, and name (**Week 3**)

This week we go deeper: asymmetric mathematics and the right mode/padding (Sections 2, 4) ·

HMAC's internal

structure and signature pitfalls (Sections 3, 5) · DH and authentication (Section 6) · PKI, X.509, and

This Week's Concepts

Each term is defined once, where it first appears in the body; here we only mark **where**.

Concept	Where
Block cipher modes, padding	Section 2
MAC, HMAC	Section 3
RSA, elliptic curve (ECC)	Section 4
Digital signature	Section 5
Diffie–Hellman (DH)	Section 6
PKI, CA	Section 7
Certificate, X.509	Section 8
Building a chain with OpenSSL	Section 9
CRL, OCSP	Section 10
HSM, PKCS#11, SoftHSM	Section 11
Post-quantum (PQC)	Section 12



1. Algorithm and Key Selection

A Brief History — Keys, Certificates, PKI

- **1976–77** — Diffie–Hellman and **RSA**: talking securely with someone you don't know
- **1988** — the **X.509** certificate format; identity carries a **CA signature**
- **1995** — commercial CAs and **PKI**; then **CRL** and **OCSP** (revocation)
- **2014** Heartbleed · **2015** Let's Encrypt · **2018** TLS 1.3 · **2022–24** **PQC** (Kyber/Dilithium)

Today's rules (the right mode/padding, chain validation, revocation) came out of these painful lessons.

Current, Standard, Properly Used

Three rules:

- **Current:** an unbroken algorithm (AES, SHA-256, Ed25519).
- **Standard:** don't write your own crypto; use a proven library.
- **Properly used:** the right mode, padding, key management.

What to Avoid

- MD5, SHA-1 (for digests), DES/3DES, RC4.
- ECB mode.
- Your own "encryption" algorithm.
- A fixed/predictable IV or key.

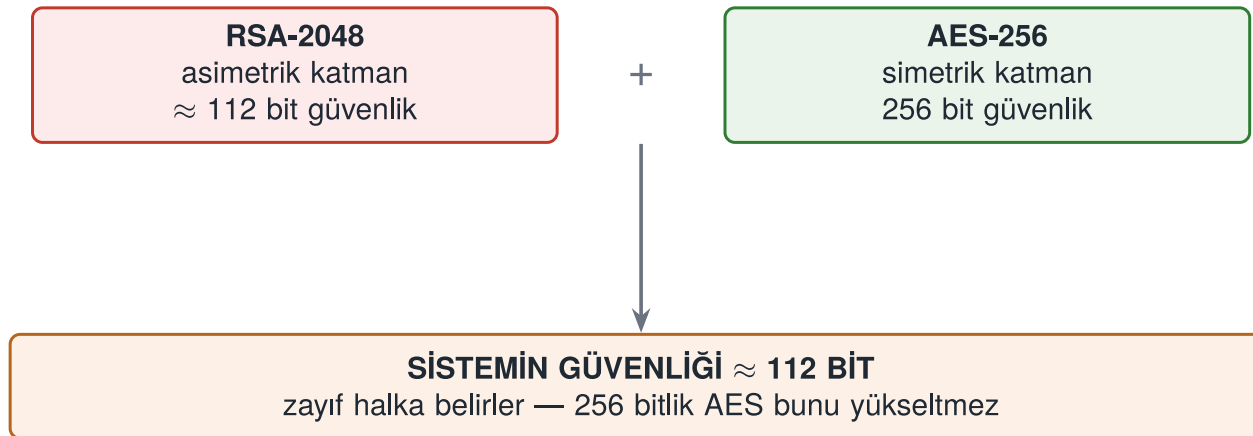
Key Length

Purpose	Recommendation (approx.)
Symmetric	AES-128 (sufficient), AES-256
RSA	≥ 2048 , prefer 3072
ECC	256-bit ($\approx$ RSA-3072)
Digest	SHA-256+

The Weakest-Link Rule

En zayıf halka kuralı

sistemin gücü, en güçlü parçası kadar değildir



Katmanları ayrı ayrı değil, en düşük güvenlik seviyesine göre değerlendirin.

- Choose components with **balanced** strength.
- Targeting 128 bits? Use RSA-3072 or ECC.

Which Mode?

Need	Choice
Confidentiality + integrity	AES-GCM / ChaCha20-Poly1305
Speed only (no hardware AES)	ChaCha20-Poly1305
Never	ECB
Legacy CBC system	+ encrypt-then-MAC, a single error

Which Asymmetric Algorithm?

Need	Choice
Signature (modern)	Ed25519
Key agreement	X25519
RSA encryption	RSA-OAEP (≥ 3072)
RSA signature	RSA-PSS

Decision Rule

- Choose current, standard, properly used.
- AEAD first; ECC first.
- Write the decision and its rationale into **S8**.



2. Block Cipher Modes and Padding

ECB · Never

- **ECB:** encrypts each block independently.
- Same block → same ciphertext block → **pattern leaks**.
- The famous "ECB penguin" example.

Don't use ECB.

Why ECB Is Bad — Diagram

ECB neden asla kullanılmaz?

ünlü “ECB pengueni” bu yüzden görünür kalır

ECB

her blok BAĞIMSIZ şifrelenir

aynı düz metin bloğu →
aynı şifreli blok
→ DESENLER görünür

CBC / CTR / GCM

her blok öncekine ya da
nonce'a bağlı

aynı metin → farklı şifreli
→ desen sızmaz

Kip seçimi şifrelemenin kendisi kadar önemlidir; varsayılan tercih AEAD (GCM).

CBC · Use with Care

- **CBC**: each block is chained with the previous one; an **IV** is required.
- The IV must be **random** and never repeated.
- Does **not provide integrity** on its own → a separate MAC is required.

CBC + Padding = Risk

- CBC requires padding.
- Handling padding incorrectly → **padding oracle** (coming up next).
- This is why the modern choice is **AEAD**.

PKCS#7 Padding · Example

Let's encrypt the 13-byte text "MERHABA DUNYA" with AES-128-CBC (K and IV are fixed here only for the demo):

```
printf 'MERHABA DUNYA' | openssl enc -aes-128-cbc -K "$K" -iv "$IV" -out cikti.bin  
xxd -p cikti.bin
```

The output is **16 bytes** (32 hex characters) — the input was 13 bytes, so **3 bytes of padding** were added.

PKCS#7 Padding · Output

Let's look at the padding content with `-nopad` (for teaching purposes only):

```
4d45 5248 4142 4120 4455 4e59 4103 0303  MERHABA DUNYA...
```

The last three bytes are `03 03 03`: the number of missing bytes ($16 - 13 = 3$) worth of bytes, each with that value, were added — exactly the PKCS#7 rule. A normal `openssl enc -d` reads and strips these bytes automatically.

PKCS#7 · Padding and the Check — Animation

data =

S	e	c	r	e	t	M	e	s	s	a	g	e	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

unpad — every check passes: the original 15 bytes are returned.

GCM · the Modern Choice (AEAD)

- **AES-GCM**: confidentiality **and** integrity together.
- **No** padding; a **nonce** (a number used once) is required.
- The nonce must **never** repeat (under the same key).

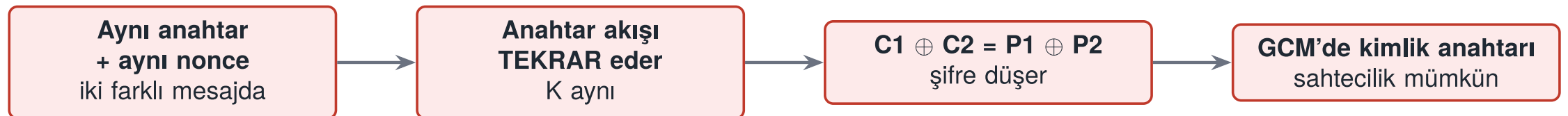
Nonce/IV Rule

- The IV/nonce must be **unique**.
- Nonce reuse in GCM is **catastrophic** (the key/data can leak).
- Use a counter or a random value (of sufficient length).

Nonce Reuse — Diagram

Nonce tekrarı neden felaket?

gizlilik VE bütünlük birlikte çöker



Nonce gizli olmak zorunda değil ama anahtar başına BENZERSİZ olmak zorundadır.

ChaCha20-Poly1305

- An AEAD alternative to AES-GCM.
- Fast when there is no hardware AES support.
- The same nonce rule applies.

Comparing the Modes — Animation

mode **ECB**

plain[] =

S	E	E	D	S	E	E	D	X	Y	A	B
---	---	---	---	---	---	---	---	---	---	---	---

ct[] =

08	62	62	BB	08	62	62	BB	77	1E	C6	D1
----	----	----	----	----	----	----	----	----	----	----	----

decrypted =

S	E	E	D	S	E	E	D	X	Y	A	B
---	---	---	---	---	---	---	---	---	---	---	---

Decrypt: everything decrypts correctly.

Modes · Edge Case

mode **GCM**

plain[] =

A	u	t	h	e	n	t	i	c	a	t	e	d	M	s	g
6F	16	21	C8	0B	B2	BD	64	37	DE	E9	90	D3	7A	85	2C

keystream =

ct[] =

2F	63	55	A0	6E	DC	C9	0D	54	BF	9D	F5	B7	37	F6	4B
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

tag=27

REJECTED — no plaintext is ever produced (fail-closed).

Padding Oracle · the Problem, Step by Step

1. While decrypting CBC, the server gives a **different** response/timing for **invalid padding** versus **invalid MAC**.
2. The attacker modifies the ciphertext and watches the responses.
3. From the response difference, they decrypt the **plaintext** byte by byte.

Why Does It Happen?

- The error message/timing **leaks** internal state.
- The attacker uses this like an **oracle**.

The Fix

- Use **AEAD** (GCM): no padding, a single verification.
- If CBC is unavoidable: **encrypt-then-MAC** + **one and the same** error.
- A timing difference is a leak too → make it constant-time.

Lesson

Encryption alone is not enough; **integrity** and a **consistent error** are required.

The padding oracle is the classic example of a "usage mistake."

Section 1–2 — Quick Check

1. Explain the weakest-link rule with an example.
2. Why is ECB never used?
3. Why is nonce reuse catastrophic in GCM?
4. How does AEAD close off the padding oracle?

Section 1–2 — Answers

1. A system is only as secure as its **weakest component**. E.g., using AES-256 but storing the key in a plain file → the key is the weakest link.
2. **ECB** turns the same plaintext block into the same ciphertext block → the pattern leaks, no semantic security.
3. The same key+nonce repeats the **keystream** (confidentiality collapses), and the GHASH **authentication key** can be recovered → catastrophe.
4. **AEAD** verifies the tag first and **refuses to decrypt** text with an invalid tag → no oracle is left for a padding error.



3. MAC, HMAC, and Integrity

Why HMAC — Diagram

Neden HMAC, neden $H(K||m)$ değil?

kendi MAC'inizi kurmayın

$H(K||m)$
Merkle–Damgård özetlerde
UZUNLUK UZATMA saldırısı
saldırgan K'yı bilmeden
 $H(K||m|| \text{ek})$ üretebilir

HMAC
iç içe iki özetleme
ipad / opad ile
uzunluk uzatma KAPALI
standart ve kanıtlı

Doğrulamayı sabit zamanlı karşılaştırmayla yapın (CRYPTO_memcmp).

Why MAC?

- Encryption gives confidentiality, **not integrity**.
- An attacker can modify the ciphertext.
- MAC: the message **has not changed** and came from the right party.

HMAC

- A MAC built on a digest function (HMAC-SHA256).
- A symmetric key; both parties know the same key.
- Verified with a **constant-time** comparison.

HMAC-SHA-256 · Real Output

Let's compute the HMAC of a payment instruction (a 32-byte key, fixed here only for the demo):

```
printf 'tutar=100;alici=TR00' | \  
openssl dgst -sha256 -mac HMAC -macopt hexkey:$ANAHTAR
```

```
SHA2-256(stdin)= a08fb115...c5f9a1
```

HMAC-SHA-256 · Avalanche Effect

Using the same key, let's change only the amount (100 → 900):

```
SHA2-256(stdin)= 8a786eac...91a46be
```

A single-character change (1 → 9) makes the tag come out **completely different** — this is called the **avalanche effect**. Without knowing the key, an attacker cannot produce a valid new tag; the receiver catches the mismatch.

HMAC · Inner/Outer Digest — Animation

key =

k	e	y	1
---	---	---	---

K' =

6B	65	79	31	00	00	00	00
----	----	----	----	----	----	----	----

K' xor ipad =

5D	53	4F	07	36	36	36	36
----	----	----	----	----	----	----	----

K' xor opad =

37	39	25	6D	5C	5C	5C	5C
----	----	----	----	----	----	----	----

message =

t	r	a	n	s	f	e	r	1	0	0	t	o	a	c	c	o	u	n	t
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

inner =

D618403D

outer (HMAC) =

09E9BA8B

outer = SHA256(K'xor_opad || inner) — the OUTER digest: this IS the HMAC tag.

Constant-Time Comparison

```
/* YANLIS: ilk farkta durur, sure farki sizdirir */  
if (memcmp(hesaplanan, gelen, 32) == 0) { /* kabul */ }  
  
/* DOGRU: her zaman 32 baytin tamamini gezer */  
if (CRYPTO_memcmp(hesaplanan, gelen, 32) == 0) { /* kabul */ }
```

Rule: always compare secret values such as a MAC/signature/password digest with a constant-time comparison; `memcmp` / `==` leaves an open door to a timing attack.

Encrypt-then-MAC — Diagram

Encrypt-then-MAC: doğru sıra

alıcı geçersiz metni hiç çözmez → dolgu kâhini kapanır



Yanlış sıra (önce çöz, sonra doğrula) dolgu hatası sızdırır ve oracle açar.

The Correct Order · Encrypt-then-MAC

```
1. şifrele:  c = ENC(k1, m)
2. MAC'le:   t = MAC(k2, c)
3. gönder:  c || t
```

- Encrypt first, **then** MAC the ciphertext.
- If the MAC does not pass, **don't even attempt** to decrypt.

Wrong Orders

- **MAC-then-encrypt**: open to the padding oracle.
- **Encrypt-and-MAC**: the MAC can leak the plaintext.
- The correct way: **encrypt-then-MAC** (or AEAD, which already does this).

Why the Order Matters — Animation

message: "SecretPayload01" (15 bytes)

S e c r e t P a y l o a d 0 1

Encrypt-then-MAC:

D1 6B A7 72 64 6F 0C 0A E8 FB 8B F9 FA 19 87 AD 6D

MAC-then-encrypt:

D1 6B A7 72 64 6F 0C 0A E8 FB 8B F9 FA 19 87 3C EB D8 2F B4 33 70 57 6C

Padding was valid; the mac IS checked and it matches: the plaintext is returned.

Replay

- The attacker resends a valid message **again**.
- The MAC is valid (the message hasn't changed), but the operation is **repeated**.
- Fix: **nonce**, timestamp, counter.

AEAD Does It All

- AES-GCM: encryption + integrity **together**.
- "Associated data" (AAD) also authenticates headers/context.
- Modern choice: AEAD instead of a separate MAC.

This Section's Rule · MAC/HMAC

- Provide integrity with **HMAC**, not plain $H(K\parallel m)$ (risk of length extension).
- When combining encryption + MAC, the order is **encrypt-then-MAC**.
- Tag/signature comparison must be **constant-time**; don't use a function with an early exit.

4. Asymmetric Cryptography: RSA and ECC

Symmetric ↔ Asymmetric — Diagram

Simetrik ve asimetrik: hangisi ne için?

pratikte HİBRİT: asimetrikle anahtar kur, simetrikle veri şifrele

SİMETRİK (AES)

tek anahtar

ÇOK HIZLI

sorun: anahtarı nasıl paylaşırsın?

ASİMETRİK (RSA/EC)

açık + gizli anahtar çifti

YAVAŞ

anahtar dağıtımı ve İMZA çözülür

TLS tam olarak bunu yapar: ECDHE ile anahtar, AES-GCM ile veri.

OAEP / PSS — Diagram

RSA dolgusu: hangisi nerede?

dolgu, algoritmanın güvenliğinin ayrılmaz parçasıdır

OAEP

RSA ile ŞİFRELEME
rastgeleleştirilmiş dolgu
ham RSA asla kullanılmaz

PSS

RSA ile İMZA
rastgeleleştirilmiş dolgu
PKCS#1 v1.5 yerine tercih

Eliptik eğride karşılıkları: X25519 (anahtar değişimi), Ed25519 (imza).

RSA · Two Jobs

- **Encryption:** encrypt with the public key, decrypt with the private key.
- **Signing:** sign with the private key, verify with the public key.
- Large keys (2048+).

RSA · Padding Is Required

- Raw RSA is **insecure**.
- **For encryption:** OAEP padding.
- **For signing:** PSS padding.
- Old PKCS#1 v1.5: avoid where possible.

Toy RSA · End to End — Animation

$$p = 61$$

$$q = 53$$

$$n = p * q = 3233$$

$$\text{phi} = (p-1)(q-1) = 3120$$

$$e = 17$$

$$d = 2753$$

$$m = 65$$

$$c = 2790$$

$$\text{decrypt}(c) = 65$$

$$\text{sign}(m) = 588$$

verify = true

verify(m, sig, pub): $\text{sig}^e \bmod n = 65; == m(65)? \text{YES} \rightarrow \text{VALID}.$

OAEP vs PSS

- **OAEP:** RSA **encryption** padding.
- **PSS:** RSA **signature** padding.
- Common mix-up: OAEP is not for signing, PSS is not for encryption.

OAEP's Randomness · Command

Let's encrypt the same message twice with OAEP:

```
openssl pkeyutl -encrypt -pubin -inkey rsa_acik.pem -in kisa.txt -out c1.bin \  
    -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256  
openssl pkeyutl -encrypt -pubin -inkey rsa_acik.pem -in kisa.txt -out c2.bin \  
    -pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256  
cmp c1.bin c2.bin && echo AYNİ || echo FARKLI
```


OAEP's Randomness · Result

```
c1.bin c2.bin differ: char 1, line 1  
FARKLI
```

Same key, same plaintext, but **different** ciphertext — OAEP mixes in a fresh random value on every encryption. **Raw RSA** (without padding) is deterministic ($c = m^e \bmod n$); that is why it is never used directly for encryption.

ECC · Why?

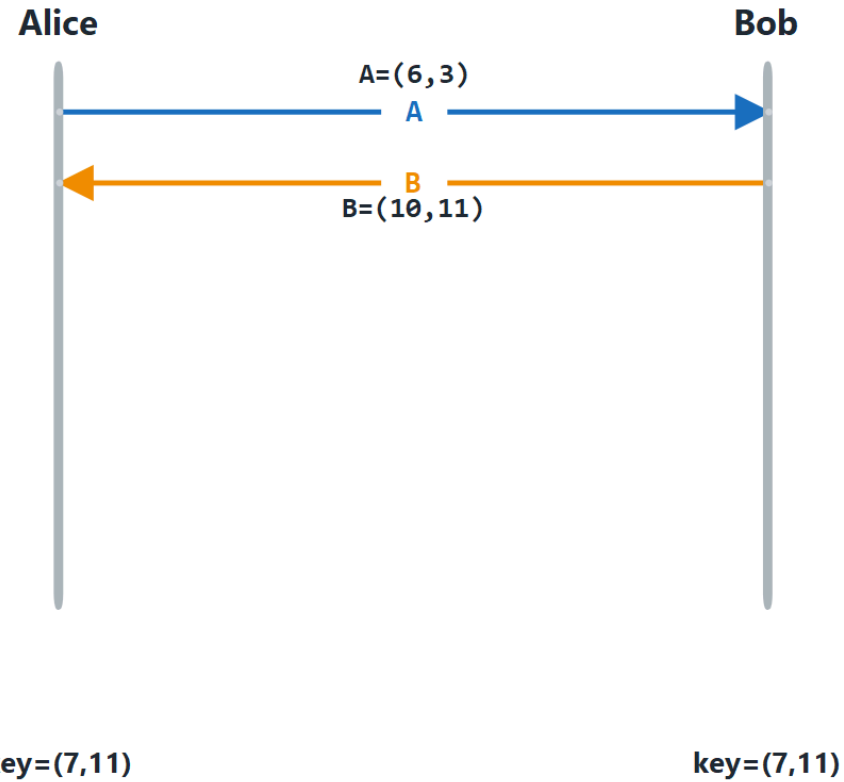
- The same security with a **smaller** key (256-bit $\approx$ RSA-3072).
- Faster, less space.
- Ideal for mobile/embedded.

Ed25519 and X25519

- **Ed25519**: the modern **signature** algorithm.
- **X25519**: modern **key agreement** (DH).
- Common mix-up: Ed25519 signs, X25519 exchanges keys.

ECDH · Point Addition — Animation

$$y^2 = x^3 + 2x + 2 \pmod{17}, G=(0,6)$$



Alice computes $a*B=(7,11)$, Bob computes $b*A=(7,11)$; the same point? YES.

RSA-3072 vs Ed25519 · Measurement

Both at roughly 128 bits of security (see the table in Section 1):

```
WC -c rsa_acik.pem ed_acik.pem  
WC -c belge.rsa.sig belge.ed.sig
```

RSA-3072 vs Ed25519 · Result

File	RSA-3072	Ed25519
Public key (PEM)	636 bytes	116 bytes
Signature	384 bytes	64 bytes

The public key is **~5.5 times** smaller, the signature **6 times** smaller. In IoT/mobile handshakes, in embedded flash, or on a blockchain, this difference adds up.

Symmetric + Asymmetric Together

1. X25519 ile ortak sır türet
2. Ondan bir AES anahtarı çıkar (KDF)
3. AES-GCM ile veriyi şifrele

- Asymmetric: carry the key. Symmetric: encrypt the data.
- TLS does exactly this.

Section 3–4 — Quick Check

1. Why is encrypt-then-MAC the correct order?
2. How is replay prevented?
3. Which is OAEP and which is PSS for?
4. What's the difference between Ed25519 and X25519?

Section 3–4 — Answers

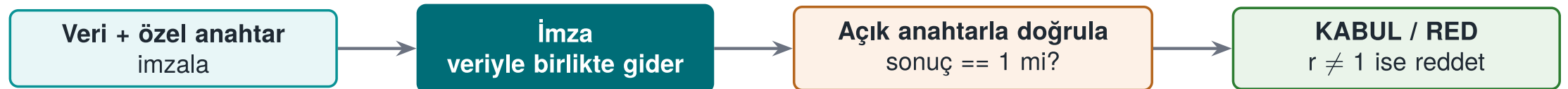
1. The receiver verifies the **MAC first**; if it fails, it does not decrypt → a modified ciphertext is never processed (closes off the padding oracle).
2. **Freshness**: a nonce/counter, a timestamp + window, a one-time challenge.
3. **OAEP** = RSA **encryption** padding; **PSS** = RSA **signature** padding.
4. **Ed25519** = signing (EdDSA); **X25519** = key exchange (ECDH). Same curve family, different job.

5. Digital Signatures and Pitfalls

Digital Signature — Diagram

Dijital imza: oluşturma ve doğrulama

en sık hata: if(r) ile denetlemek



API 1=başarı, 0=başarısız, <0=hata döner; if(r) hatayı da “doğru” sayar.

Signature · What Does It Provide?

- **Integrity:** the message has not changed.
- **Identity/non-repudiation:** the private key holder signed it.
- **Verification:** with the public key, anyone can do it.

Signature · Where Is It Used?

- Signing software/updates.
- Certificates (CA signature).
- Signing documents/transactions.
- Server identity in TLS (CertificateVerify).

Pitfall 1 · Not Checking the Return Value

```
int r = EVP_DigestVerify(ctx, imza, n, veri, m);  
if (r) guncellemeyi_kur(); /* HATALI */
```

A negative error value is also treated as "true." The correct check is `r == 1`.

Pitfall 2 · Not Including the Version in the Signature

- If the signature covers only the content, an attacker can install an **old** but validly signed version (downgrade).
- The signed content must also cover the **version number**; an old version must be rejected.

Pitfall 3 · Verifying with the Wrong Key

- Is the verification key trusted? Where did it come from?
- If you verify with the attacker's key, their signature is "valid."
- The key must be **pinned**, or come from a trusted chain.

Signature Verification · Three Scenarios — Animation

document = U p d a t e P a c k a g e 0 1

sig=1287

verify(signer.pub) = true

```
openssl dgst -verify signer.pub -signature ... — result: VALID.
```

Signature · the Digest Rule

- A signature is really the signing of a **digest**.
- A weak digest (MD5/SHA-1) → a weak signature.
- Use SHA-256+.

6. Diffie–Hellman and MITM

DH · What Does It Do?

- Two parties derive a shared secret **without sharing** a private key.
- Key agreement over the network.
- The modern form: X25519.

DH · Step by Step (Concept)

Ali: a gizli, $A = g^a$ açık gönderir
Veli: b gizli, $B = g^b$ açık gönderir
Ortak sır: $A^b = B^a = g^{(ab)}$

An eavesdropper sees g^a and g^b but cannot compute $g^{(ab)}$.

X25519 · Both Sides Reach the Same Secret

```
openssl genpkey -algorithm X25519 -out alice.key
openssl genpkey -algorithm X25519 -out bob.key
openssl pkey -in alice.key -pubout -out alice.pub
openssl pkey -in bob.key -pubout -out bob.pub

openssl pkeyutl -derive -inkey alice.key -peerkey bob.pub -out alice_sir.bin
openssl pkeyutl -derive -inkey bob.key -peerkey alice.pub -out bob_sir.bin
```

X25519 · Result

```
8c53744a000c1a6f...bbc1376  
8c53744a000c1a6f...bbc1376  
AYNI
```

Alice and Bob, without ever knowing each other's private key, reached the **same** 32-byte secret by exchanging only public keys over the network. This raw secret is not used directly as an AES key; it is first passed through a **KDF** (HKDF) to derive session keys.

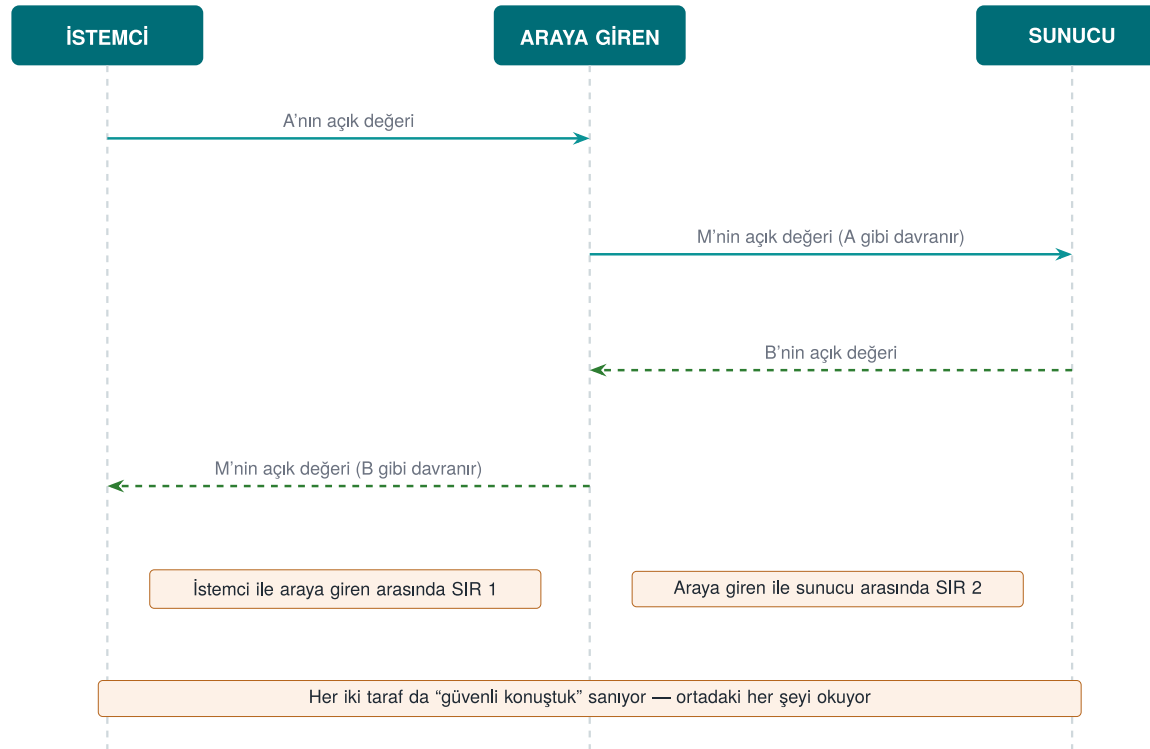
Unauthenticated DH · MITM Risk

- If Alice and Bob don't **verify** each other's identity...
- An attacker **sits in the middle**: doing DH separately with Alice and separately with Bob.
- Listening to/modifying both.

Unauthenticated DH · MITM — Diagram

Kimliksiz Diffie–Hellman: araya girme

DH kiminle anlaştığınızı SÖYLEMEZ



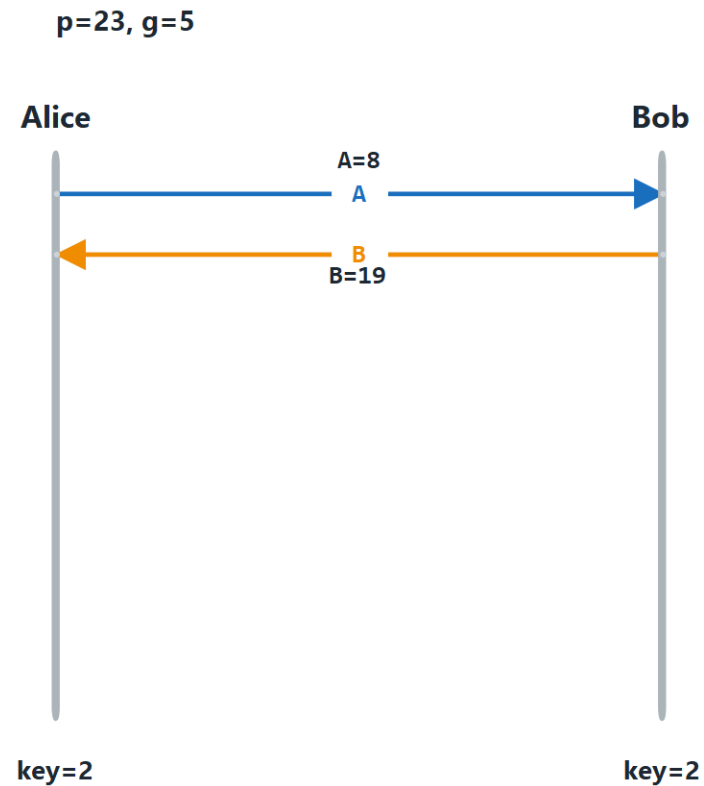
Çözüm: DH'yi kimlik kanıtıyla bağla (sertifika / imzalı DH) — TLS 1.3 bunu yapar.

MITM · Diagram

Alice ↔ [Attacker] ↔ Bob
two separate DH exchanges; the attacker sits in the middle

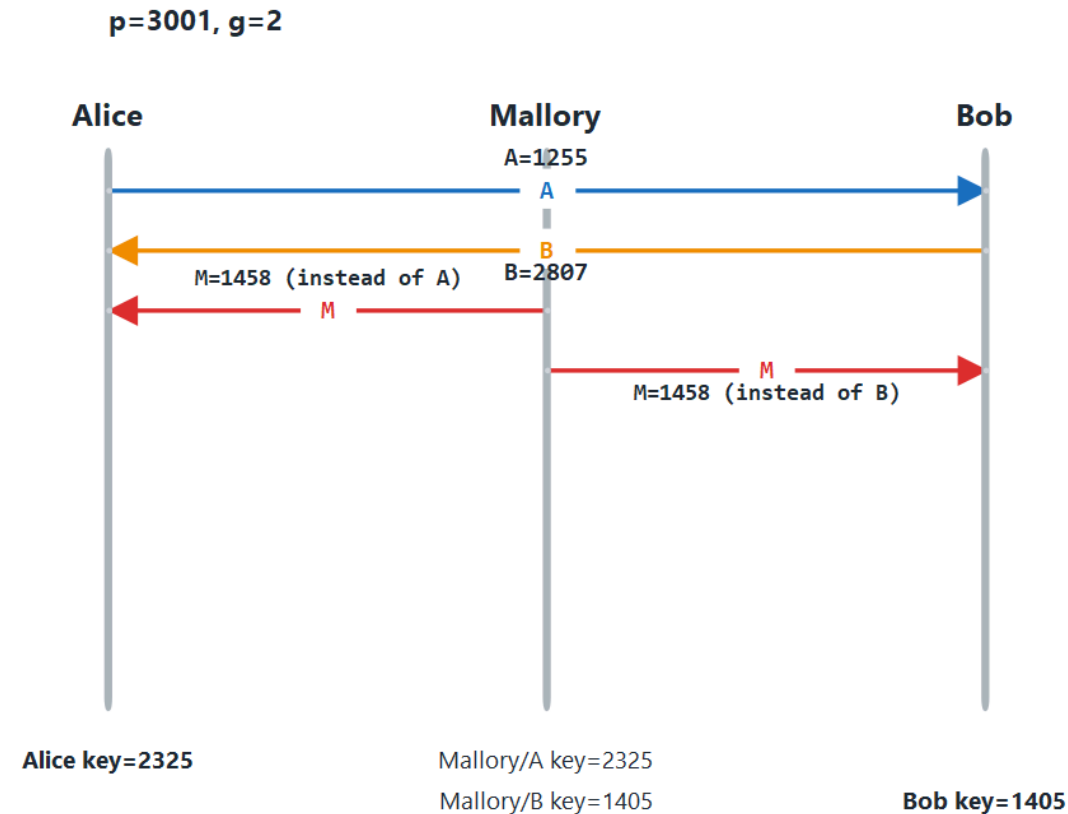
Both sides think "I've established a secure channel."

DH and MITM — Animation



shared_secret(...) — Alice $B^a \bmod p = 2$, Bob $A^b \bmod p = 2$; equal? YES.

DH/MITM · Edge Case



Is Alice's key (2325) DIRECTLY equal to Bob's key (1405)? NO — both are really talking to Mallory.

The Fix · Authenticated DH

- The DH values are **signed** with a key whose identity is known.
- `CertificateVerify` in TLS 1.3.
- The other side **verifies** this identity (certificate chain).

Forward Secrecy

- A **new** DH key for every session.
- Even if the long-term key leaks, **old** sessions cannot be decrypted.
- Modern TLS provides this.

Section 5–6 — Quick Check

1. Why is `if (r)` wrong in signature verification?
2. Why must the version be part of the signature?
3. How does MITM happen against unauthenticated DH? The fix?
4. What does forward secrecy provide?

Section 5–6 — Answers

1. Verification returns 1=success, 0=failure, <0=error; `if (r)` also treats the error (–1) as "true." Check for `r == 1`.
2. Otherwise an attacker can present an **old/insecure version** as "valid" with the same signature (downgrade). The version must be part of the signed data.
3. The attacker establishes a separate key with each side. Fix: bind DH to **authentication** (signed DH / a certificate).
4. Each session uses an **ephemeral** key; even if the long-term key leaks, **past sessions** cannot be decrypted.

7. Public Key Infrastructure (PKI)

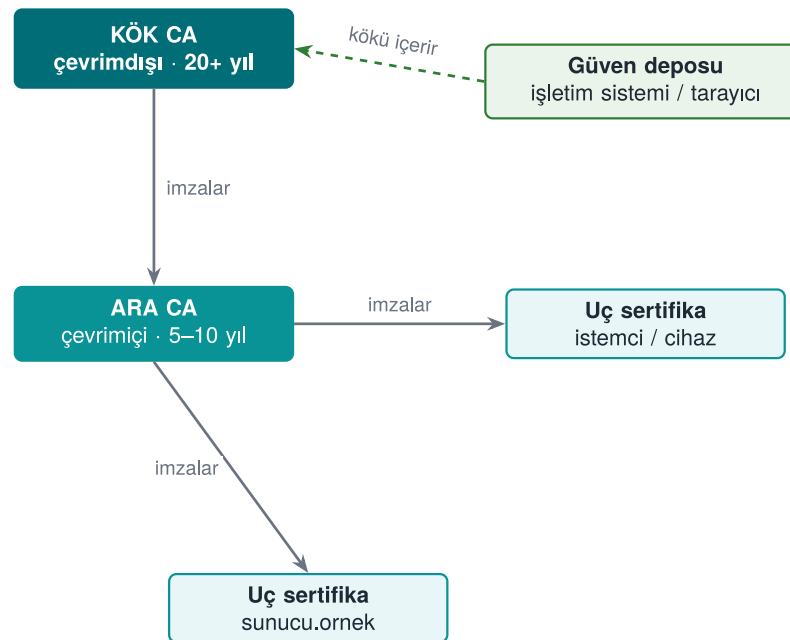
The Problem · Who Do We Trust?

- You've seen a public key. Does it really belong to that person/site?
- A malicious actor can present their own key as "the bank."
- PKI solves this trust problem.

Chain of Trust

PKI: kim kime güvenir?

güven kökten uca akar



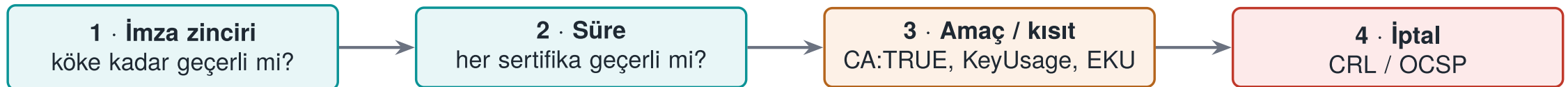
İstemci yalnız KÖKE güvenir; zincir kökten uca kesintisiz kurulamazsa doğrulama başarısızdır.

Each level **signs** the one below it.

Chain Validation — Diagram

Zincir doğrulamanın dört sorusu

+ ad denetimi (SAN) — beşi birden geçmeden kabul edilmez



Eksik ara sertifika zinciri kurdurmaz; fail-open ile “geçti” saymak en sık hatadır.

Root and Intermediate CA

- **Root CA:** offline, self-signed, heavily protected.
- **Intermediate CA:** issues day-to-day certificates.
- If the root leaks, it's a **catastrophe**; this is why an intermediate CA is used.

Trust Store

- The OS/browser carries the trusted **root CAs**.
- If a chain reaches one of these roots, it is trusted.
- If it can't, it is **untrusted**.

Four Questions

When validating a certificate chain:

1. Is the **signature** valid? (at every level)
2. Does the **chain** reach a trusted root?
3. Has the **validity period** not expired?
4. Does the **name** match (SAN)?

All four must be **yes**.

Question 1 · Chain (Issuer/Subject)

```
openssl x509 -in sunucu.crt -noout -issuer -subject  
openssl x509 -in ara.crt -noout -issuer -subject  
openssl x509 -in kok.crt -noout -issuer -subject
```

```
issuer=CN=...Ara CA    subject=CN=localhost  
issuer=CN=...Kok CA    subject=CN=...Ara CA  
issuer=CN=...Kok CA    subject=CN=...Kok CA
```

The chain reads: `sunucu` 's issuer = `ara` 's subject; the root signs **itself**.

Question 2 · Validity

```
openssl x509 -in sunucu.crt -noout -dates  
openssl x509 -in sunucu.crt -noout -checkend 0
```

```
notBefore=Sep 23 2026 GMT  
notAfter=Dec 22 2026 GMT  
Certificate will not expire
```

-checkend 0 : "has it expired as of right now?" TLS clients ask this automatically on every connection.

Question 3 · Usage (CA:TRUE/FALSE)

```
openssl x509 -in sunucu.crt -noout -ext basicConstraints,keyUsage  
openssl x509 -in ara.crt -noout -ext basicConstraints,keyUsage
```

```
sunucu.crt:  CA:FALSE          (baska sertifika imzalayamaz)  
ara.crt:     CA:TRUE, pathlen:0 (yalniz uc sertifika imzalayabilir)
```

Question 4 · Name (SAN)

```
X509v3 Subject Alternative Name:  
DNS:localhost, IP Address:127.0.0.1
```

The client compares the address it connected to against this list — **not** the `CN` in `Subject` .

A client library asks these four questions **on your behalf**; turning off `verify` or swallowing errors means none of the four questions are ever asked.

With OpenSSL · Example

```
openssl verify -CAfile kok.crt \  
-untrusted ara.crt sunucu.crt
```

- `-CAfile` : the trusted root.
- `-untrusted` : the intermediate certificate(s).

SPKI Pinning

- The application embeds the **digest** of the expected server key.
- Even a fake but "valid" certificate is not accepted.
- A **backup pin** is required (so you're not locked out when the key changes).

This Section's Rule · Chain Validation

- Don't skip any of the chain validation's **four questions** (signature, validity, usage, name).
- `verify` does not check the name — **SAN** checking must be done separately.
- Even a cryptographically correct chain can't connect if the server doesn't send the **intermediate certificate**.



8. The X.509 Certificate

X.509 Fields — Diagram

X.509 sertifikasında ne var?

doğrulama bu alanların HEPSİNİ denetler

Konu (Subject) + SAN — kime ait — ad denetimi SAN'a bakar, CN'e değil

Açık anahtar (SPKI) — pinning bunu sabitler

Geçerlilik aralığı — notBefore / notAfter

Uzantılar — basicConstraints (CA:TRUE), KeyUsage, EKU

CA imzası — zincirin bir üst halkası tarafından

Yalnız “imza geçerli” demek yetmez: ad, süre, amaç ve iptal de denetlenir.

What's Inside?

- **Subject:** who it belongs to (domain name).
- **Public key.**
- **Validity:** start/end.
- **Issuer:** which CA signed it.
- **Signature.**

SAN · Name Checking

- **SAN (Subject Alternative Name):** the domain names the certificate is valid for.
- Name checking is done against the **SAN, not the CN**.
- A certificate for `example.com` must not be valid for `baska.com`.

Real Certificate · Top-Level Fields

```
Version: 3 (0x2)
Serial Number: 35:e2:f6:8d:....:76:e7
Signature Algorithm: ecdsa-with-SHA256
Issuer: CN=CEN429 Lab Ara CA
Validity: Sep 23 2026 - Dec 22 2026
Subject: CN=localhost
Subject Public Key Info: 256 bit, NIST CURVE: P-256
```

Real `openssl x509 -text` output; each field carries the answer to one of the "four questions" (next section).

Real Certificate · Extensions

```
X509v3 Basic Constraints: CA:FALSE
X509v3 Key Usage: critical, Digital Signature
X509v3 Extended Key Usage: TLS Web Server Authentication
X509v3 Subject Alternative Name:
    DNS:localhost, IP Address:127.0.0.1
X509v3 Authority Key Identifier: F5:BA:39:86:...
```

The `Authority Key Identifier` is the **fingerprint** of the CA that signed the certificate; when building the chain, it prevents confusion with a "fake intermediate CA with the same name but a different key."

9. Building a Certificate Chain with OpenSSL

Goal

Let's build a small chain:

Root CA → Intermediate CA → Server certificate.

Then let's validate it with `verify` and see the typical mistake.

Step 1 · Root CA (Self-Signed)

```
openssl req -x509 -newkey ed25519 \  
  -keyout kok.key -out kok.crt \  
  -subj "/CN=Ders Kok CA" -days 3650 -nodes
```

The root signs itself; it is kept offline.

Step 2 · Intermediate CA (Signed by the Root)

```
openssl req -newkey ed25519 -keyout ara.key \  
  -out ara.csr -subj "/CN=Ders Ara CA" -nodes  
openssl x509 -req -in ara.csr -CA kok.crt -CAkey kok.key \  
  -CAcreateserial -out ara.crt -days 1825
```


Step 3 · Server Certificate (Signed by the Intermediate)

```
openssl req -newkey ed25519 -keyout sunucu.key \  
    -out sunucu.csr -subj "/CN=ornek.test" -nodes  
openssl x509 -req -in sunucu.csr -CA ara.crt -CAkey ara.key \  
    -CAcreateserial -out sunucu.crt -days 365
```

Step 4 · Validate (Missing Intermediate)

```
openssl verify -CAfile kok.crt sunucu.crt  
# HATA: unable to get local issuer certificate
```

The chain can't reach the root: the **intermediate certificate is missing**.

Step 5 · Validate (With the Intermediate)

```
openssl verify -CAfile kok.crt \  
    -untrusted ara.crt sunucu.crt  
# OK
```

Given the intermediate certificate, the chain is complete.

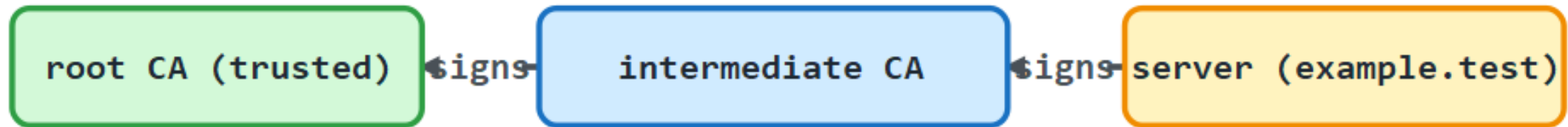
Lesson

- The server must also send the **intermediate** certificate.
- If it's missing, the client says "issuer not found."
- The most common TLS configuration mistake in the field.

The Chain's Four Questions · in Practice

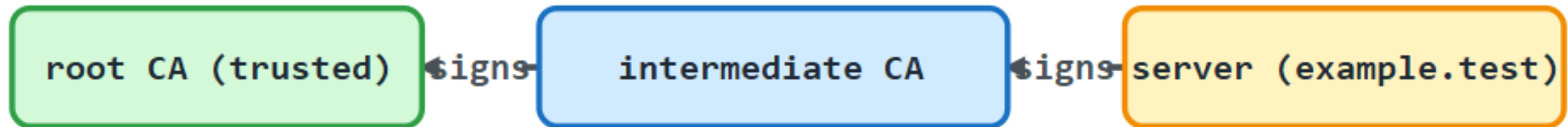
- **Signature:** each level signed by the one above ✓
- **Root:** `-CAfile kok.crt` trust store ✓
- **Validity:** within `-days` ✓
- **Name:** SAN checking in the application (verify does not check the name!)

Chain Validation · Six Scenarios — Animation



`-verify_hostname example.test` — the SAN matches: the CHAIN IS ACCEPTED.

Chain · Edge Case (Expired)



-atime — at the check time, the server certificate has EXPIRED.

Turning Off Certificate Validation

```
/* ASLA: */  
SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL);
```

This opens the door to MITM. A line left in "for testing" is a catastrophe in the field.

Skipping Name Checking

- If the chain is valid but the **name** is not verified, the attacker's valid certificate is accepted.
- The hostname (SAN) must always be checked.

Critical Reading Rule

Ask this when reading TLS setup code:

- Is validation turned on?
- Is the name checked?
- Is it **fail-closed** on error?



10. Certificate Revocation

Certificate Revocation — Diagram

Sertifika iptali: CRL · OCSP · zımbalama

“bu sertifika hâlâ geçerli mi?”



FAIL-OPEN TUZAĞI: “erişilemedi” = “geçti” sayma!

Why Revoke?

- If a certificate's private key leaks, it must be **revoked** before it expires.
- A revoked certificate must not be accepted.

CRL

- **CRL:** a **periodic list** of revoked certificates.
- Can be large, and can be out of date.

OCSP

- **OCSP**: asking about a certificate's status **on the spot**.
- More current; but has privacy/speed issues.

OCSP Stapling

- The server fetches the OCSP response **itself** and presents it along with the certificate.
- Gains privacy + speed.

CRL Workflow · Revoke and Generate

```
openssl ca -config ara.cnf -revoke sunucu.crt  
openssl ca -config ara.cnf -gencrl -out ara.crl
```

```
Revoking Certificate 35E2F68D...76E7.  
Database updated
```

The certificate file **does not change**; only a revocation record is added to the CA's ledger (`index.txt`). `-gencrl` turns this record into a list **signed** with the CA's own key.

Validating with a CRL · Result

```
openssl verify -crl_check -CAfile kok.crt \  
-untrusted ara.crt -CRLfile ara.crl sunucu.crt
```

```
error 23 at 0 depth lookup: certificate revoked  
error sunucu.crt: verification failed
```

Even though the certificate **has not yet expired**, it is rejected because it is listed in the CRL — revocation checking works **independently** of validity checking.

CRL vs OCSP — Animation

certificate, serial #4711 issued, not revoked

OCSP: `ocsp_query(#4711) = "good"`

CRL: `crl_check(#4711) = "good"`

`crl_check(4711, crl, at_time)` — the CRL is still fresh, result: "good".

The Fail-Open Trap

- **Accepting** a certificate when OCSP is unreachable = fail-open.
- An attacker can block OCSP and slip a revoked certificate through.
- Mitigation: **Must-Staple**, short-lived certificates.

Section 7–10 — Quick Check

1. What are the four questions of chain validation?
2. What is name checking done against (CN or SAN)?
3. Why does a missing intermediate certificate break `verify` ?
4. What is fail-open, and how is it mitigated?

Section 7–10 — Answers

1. (1) is the chain **valid** to the root, (2) is it within its **validity** period, (3) does it have the right **purpose/constraint** (CA:TRUE, KeyUsage/EKU), (4) has it been **revoked** (CRL/OCSP).
2. **SAN** (Subject Alternative Name); CN is no longer used.
3. The chain **cannot be built** to the root; the trust path is never completed → failure. The server must also send the **intermediate** certificate.
4. **Fail-open**: counting an error/unreachability as "passed." Mitigation: **fail-closed**, OCSP stapling / must-staple, reject on error.



11. Storing Keys in Hardware

What Is an HSM?

- **HSM:** dedicated hardware that stores/operates keys.
- The key never leaves the HSM; you say "sign this," and the result comes back.
- Tamper-resistant.

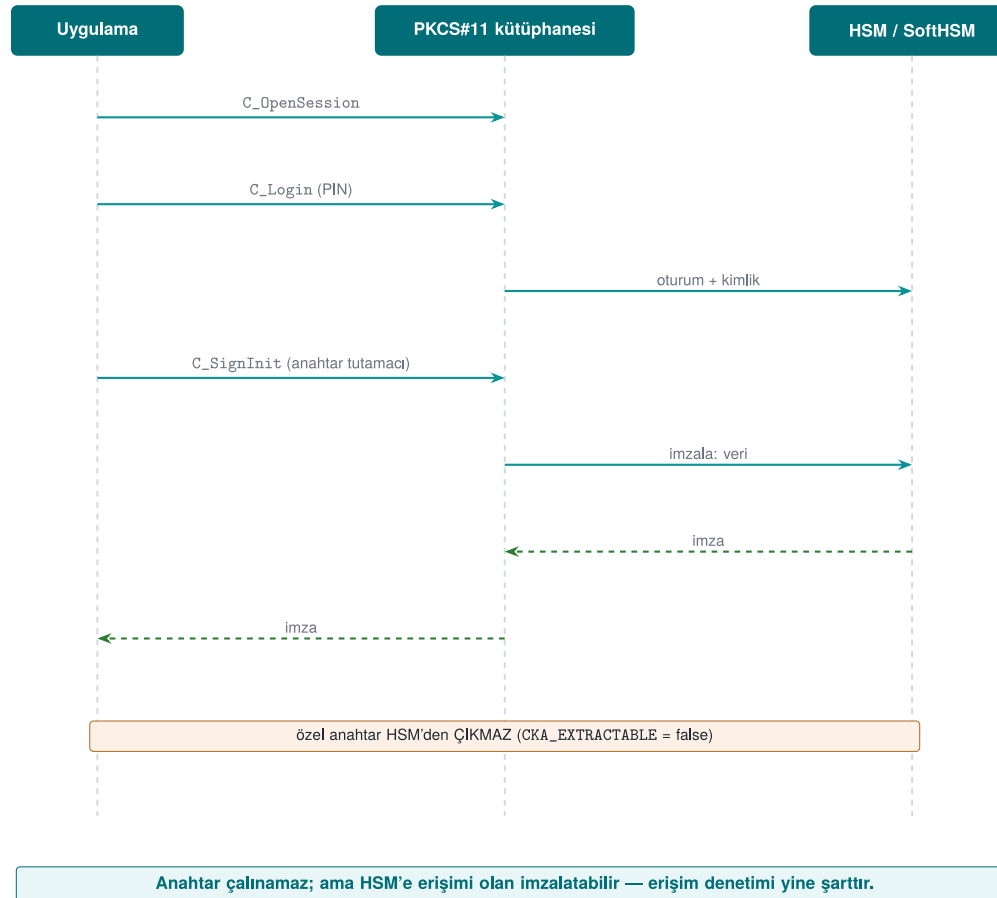
PKCS#11

- The **standard interface** for talking to key modules.
- The application never **sees** the value of the key.
- HSMs and software simulations expose this same interface.

Signing with PKCS#11 — Diagram

PKCS#11 ile imzalama: anahtar hiç dışarı çıkmaz

uygulama anahtar değil, anahtarın TUTAMACINI görür



The Key Never Leaves — Animation

HSM (key NEVER leaves)

handle #1 generated

document =

P u r c h a s e o r d e r 0 1

sig=36976267

verify(1, document, sig) = true

verify(1, document, sig) — the same handle: the signature verifies.

SoftHSM

- A **software simulation** of an HSM; the same PKCS#11 interface.
- For development/testing; **no real hardware protection**.
- A real HSM in production.

Why Hardware?

- In software, the key is exposed to a whitebox attacker (Week 11).
- In hardware, the key is isolated → far stronger.
- If possible, never put the key in software at all.

Key Lifecycle

Stage	What's Done
Generation	Strong randomness (CSPRNG)
Storage	HSM/TEE or protected
Use	Least privilege, constant-time
Crypto-period	A lifespan limit
Renewal	Regular
Destruction	Secure erasure from memory

This Section's Rule · Key Storage

- When generating a valuable key, `CKA_EXTRACTABLE=false` must be set **explicitly**; don't rely on the default.
- On an HSM the key never leaves; the application only says "sign" through a **handle**.
- SoftHSM is for testing; production requires a real HSM/hardware protection.



12. Post-Quantum Cryptography

Post-Quantum — Diagram

Kuantum sonrası: neyi değiştiriyor, neyi değiştirmiyor?

NIST seçimi: ML-KEM (Kyber) ve ML-DSA (Dilithium)

TEHDİT ALTINDA

RSA, DH, ECDH, ECDSA

“şimdi topla, sonra çöz”

bugünkü trafik ileride çözülebilir
geçiş bugün planlanır

BÜYÜK ÖLÇÜDE GÜVENLİ

AES-256 (anahtar boyu yeter)

SHA-384 / SHA-3

simetrik dünya ayakta
yalnız anahtar boyu artar

Uzun ömürlü sırlarınız varsa hibrit anahtar değişimine geçiş bugünün kararıdır.

The Threat

- A sufficiently powerful quantum computer could break today's **RSA/ECC**.
- The "harvest now, decrypt later" attack: store encrypted data today, decrypt it in the future.

PQC · Approach

- Quantum-resistant algorithms (e.g., ML-KEM key encapsulation).
- Standardization is ongoing.
- **Crypto agility:** a design that lets you swap the algorithm easily.

PQC Sizes · Command

OpenSSL 3.5 can generate the standardized PQC algorithms **today**:

```
openssl genpkey -algorithm ML-KEM-768 -out mlkem.pem
openssl genpkey -algorithm ML-DSA-65 -out mldsa.pem
openssl pkeyutl -sign -inkey mldsa.pem -rawin -in belge.txt -out belge.mldsa.sig
wc -c mlkem_pub.pem mldsa_pub.pem belge.mldsa.sig
```

PQC Sizes · Result

Algorithm	Public Key	Signature
Ed25519 (classical)	116 bytes	64 bytes
RSA-3072 (classical)	636 bytes	384 bytes
ML-KEM-768 (PQC)	1,714 bytes	—
ML-DSA-65 (PQC)	2,770 bytes	3,309 bytes

ML-DSA-65: the public key is **~24 times** larger than Ed25519's, the signature **~52 times** larger. TLS's **hybrid** (classical + PQC) approach exists to manage this transition cost.

What Should You Do Today?

- Be crypto-agile (don't hardcode the algorithm).
- Watch PQC for long-lived secrets.
- No panic; but prepare.



13. Project: This Week (S8, S11)

Project · S8/S11

- [] **Algorithm inventory:** purpose, algorithm, mode, key length, library.
- [] A **key lifecycle** table.
- [] TLS/certificate validation; pinning + backup pin if applicable.
- [] Signature verification (on the update file).

End to End: a Crypto Design Case Study

A mobile app:

- Encrypts sensitive data locally.
- Talks to a server securely.
- Verifies updates.

Let's design the crypto.

Local Data Encryption

- **AES-256-GCM** (AEAD): confidentiality + integrity.
- A counter or random nonce, **never repeated**.
- The key in TEE/HSM; otherwise whitebox (Week 11).

Server Communication

- **TLS 1.3:** X25519 key agreement + AEAD.
- Certificate chain validation + (if applicable) SPKI pinning + a backup pin.
- Not fail-open.

Update Verification

- An **Ed25519** signature; `verify == 1`.
- The signed content covers the **version**; downgrade rejected.
- The verification key is pinned/embedded (protected by integrity).

Key Inventory (S8)

Key	Purpose	Alg/Length	Storage
Data key	Local encryption	AES-256	TEE/whitebox
Session key	TLS	X25519-derived	Memory, short-lived
Signature verification	Update	Ed25519 public	Embedded

Classic Crypto Mistakes — Summary

Mistake	Section	The Right Way
Writing your own crypto	1	A proven library, a standard algorithm
ECB mode	2	AEAD (GCM / ChaCha20-Poly1305)
IV/nonce reuse	2	A unique, non-repeating nonce
Encryption without integrity	3	AEAD or encrypt-then-MAC
Raw RSA / wrong padding	4	OAEP (encryption), PSS (signing)
Weak digest	5	SHA-256+
Not checking the signature return value	5	<code>== 1</code> , and the version is covered
Not checking the chain/name	7	The four questions + SAN checking
Fail-open revocation	10	Must-Staple, short lifetimes
Embedding the key in plaintext	11	HSM/TEE; otherwise whitebox + a layer (Week 11)

All ten mistakes were covered earlier in this deck; here they are gathered in one glance.



Solved Self-Check

Question 1

What is the security level of an RSA-2048 + AES-256 system?

Answer: ~112 bits (RSA-2048 is the weakest link). For 128 bits, use RSA-3072 or ECC.

Question 2

"Invalid padding" and "invalid MAC" as separate messages: what's the risk?

Answer: A padding oracle; the attacker can decrypt the message by watching the responses. Use one and the same error; prefer AEAD.

Question 3

Why is `if (EVP_DigestVerify(...))` wrong?

Answer: A negative error value also counts as true; the check should be `== 1`. Also, the signed content must cover the version.

Question 4

Why does `verify` fail without the intermediate certificate?

Answer: The chain can't reach the root; the intermediate certificate is missing. In the field, this usually means the server isn't sending the intermediate certificate.

Question 5

What happens when `openssl verify` is given both a correct and an irrelevant intermediate certificate?

Answer: OpenSSL builds the chain along any valid path it can (e.g., directly to the root); the irrelevant certificate just remains an unused candidate, and validation can still succeed. This is an operational, not a cryptographic, situation.

Question 6

Why is reusing the same random value in two ECDSA signatures a disaster?

Answer: The private key can be computed from the two signatures. Ed25519 or deterministic ECDSA (RFC 6979) removes this risk.

Question 7

Why is the root CA kept offline, and why does an intermediate CA exist?

Answer: Compromise of the root key collapses the entire chain, and removing the root from trust stores takes years. The intermediate CA does the day-to-day work; if it is compromised, only that one is revoked.

Question 8

Why shouldn't the key be placed in software? Alternative?

Answer: A whitebox attacker can extract a key embedded in software. Alternative: HSM/TEE (PKCS#11); otherwise whitebox + a layer (Week 11).

Glossary

Term	Meaning
AEAD	Encryption + integrity together
Encrypt-then-MAC	The correct order
OAEP/PSS	RSA encryption/signature padding
Ed25519/X25519	Signature / key agreement
SAN	Certificate name checking
CRL/OCSP	Revocation list / on-the-spot query

Glossary (Continued)

Term	Meaning
Nonce	A number used once
KDF	Key derivation function
Forward secrecy	Old sessions can't later be decrypted
Trust store	Trusted root CAs
Crypto-period	A key's lifespan
Crypto agility	Easily swapping the algorithm

Next Week

Week 11 — Whitebox Cryptography

This week we saw the "entrust it to hardware" path for protecting a key (HSM, PKCS#11, `CKA_EXTRACTABLE=false`, Section 11). Week 11 asks the same question with no hardware available: how do you protect a key inside an application that is pure software, with no access to any HSM? Table-based whitebox AES and the attacks built against it (BGE, DCA, DFA) are the software-side counterpart of this week's "the key must never be exposed" principle.

This week in one sentence: in cryptography, security lies less in the right algorithm than in the right

usage — AEAD, the right padding, a verified signature, a checked chain, a well-managed key.

Judge

cryptography not by "on/off," but by whether it is used correctly.